

Edge Coffee Machine

Smart Coffee Machine featuring Edge AI

Version	0.x
Author - Date	Samuel Mira – 30.7.2025
Verified by – Date	... –
Approved by – Date	... – ...
Status	Draft
Distribution	Qt, Mälardalen University, Politecnico di Milano, University of Zagreb
Location	Berlin, Germany, World
Comments	Initial project proposal

Table of contents

1. Introduction..... 4

2. Customer story 4

3. Expected outcome..... 5

4. Scope..... 6

4.1. Product aspect..... 6

4.1.1. Customer recognition 6

4.1.2. Customer classification 6

4.1.3. Drinks offer..... 7

4.1.4. Brewing process..... 7

5. Suggested technical implementation 7

5.1. Frontend..... 8

5.2. Backend 8

5.3. AI Component..... 8

1. Introduction

The Qt Company¹ is a software company mainly known for its open-source user interface (UI) framework. Qt for MCU is a commercial product that also supports Microcontrollers (MCU). Qt provides complete graphics framework and toolkit to create and deliver smartphone-like user experiences on resource-constrained devices. It is used across many industries and can be found in consumer appliances, motorcycles or even professional digital cameras. It is used on Ducati motorcycles², Hasselblad³ cameras and B/S/H⁴ home appliances.

The MCU market is very competitive, product complexity is growing and at the same time there's a push to by lower production prices. A lot of new devices are also integrating Artificial Intelligence (AI) capabilities. Features like voice assistant, intelligent customization, remote control, and life personalization are just a few examples of those. Currently, the architecture of those devices and applications often requires internet connection and integration with 3rd party APIs from Google, OpenAI and others.

The complexity of these solutions is troubling, as they are costly to maintain with necessary development, server and application maintenance. They can be affected by several situations outside of the customer or manufacturer control, such as internet availability or data protection and processing. Finally, it also requires very powerful and power-hungry hardware.

A possible solution for those issues is Edge AI. This refers to the implementation of machine learning algorithms directly within small, low-power embedded devices. Edge AI on MCUs offers lower operational costs, reduced power consumption, enhanced data security (due to no cloud dependency) and improved response times. Currently MCU vendors are pushing into this direction with the release of more powerful MCUs, and in some cases, adding NPUs (Neural Processing Unit) to existing MCU products.

2. Customer story

Imagine a scenario - it's Monday and you just arrived to the office at 7:55AM since you have a meeting at 8AM. What kind of person schedules a meeting at 8AM? Everything is pain and hardship. You can barely see but coffee machine is your friend. It knows who you are and what your favourite drink is, it understands you but doesn't judge. It can't give you a hug but it can quickly brew you a cup of warm and fresh cup of your preferred beverage, helping you survive that meeting and come back to the land of the living.

¹ <https://www.qt.io>

² <https://www.qt.io/ducati-built-with-qt>

³ <https://www.qt.io/hasselblad-built-with-qt>

⁴ <https://www.qt.io/bsh-built-with-qt>

3. Expected outcome

Your task will be to create the backend and user interface of the mentioned coffee machine. It will be an application that is built for a microcontroller board referenced in *Suggested technical implementation* section.

The application needs to access the provided camera and capture an image or a video of the customer standing in front of the coffee machine. The picture is analysed using machine learning to recognize the user. Recognized customer is presented by a customized set of drinks compared to the default one. Customer can, using voice prompt, to ask the coffee machine to brew the selected choice.

UI is showing the user if they have been recognized by the machine learning system. Additionally, UI allows selecting a drink from the list using touchscreen. Drinks can be customized. User interaction is simple and fluid.



Figure 1 – Allespresso concept design

* *Allespresso concept design* - <https://www.behance.net/gallery/206658869/Allespresso-Coffee-Maker-UIUX-Design>

* *Qt for MCU, currently features a coffee machine demo, which can be used as a starting point*

4. Scope

4.1. Product aspect

4.1.1. Customer recognition

Coffee machine automatically attempts to recognize the customer. Based on the result, customer is categorized as a new or already known one. New customers get a default selection on drinks. As they use the machine, machine learns about them and provides more personalized options.



Figure 2 - Customer recognition

4.1.2. Customer classification

Customers are classified in one of 3 groups:

- Default customers
 - o Users that system doesn't know enough about
 - o Gets the list of most popular drinks
- Conservatives
 - o Users that always select the same drink or two and don't customize it
 - o Their preferred drink or two is shown first and largest
- Early adopters
 - o Users that often try new drinks, alternate choices and adjust values
 - o They are shown a drink they haven't tried yet and are shown customization options

4.1.3. Drinks offer

User interface offers a selection of at least 10 possible drinks served by the coffee machine. Each of the drinks has a distinct graphic. User can select one of the drinks using touchscreen. Each of the drinks has a default preset specifying the brewing process – how much coffee beans, cocoa powder, water, foam and milk. It's possible to adjust the amount of each of the ingredients. If a drink doesn't include some ingredients, it should not be shown available for modification.



Figure 4 - Drink selection screen

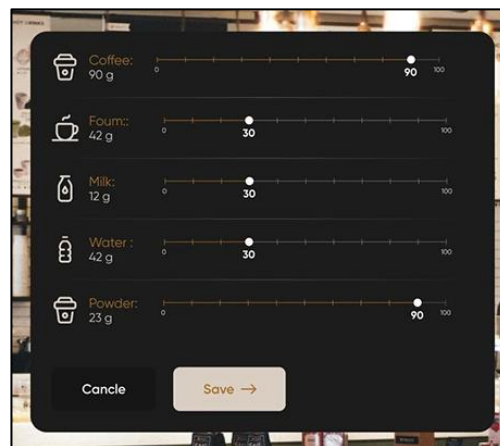


Figure 3 - Drink customization screen

4.1.4. Brewing process

After customer selects a drink, they click on brew button. User interface shows a brewing process that is shorter or longer depending on the drink and brewing process customization. Brewing process can be interrupted. After brewing, or interruption, coffee machine goes into default state, waiting for the next customer.

5. Suggested technical implementation

The vendor will be from the popular Espressif Systems. Espressif is one of the largest microcontroller suppliers globally, having shipped over 1 billion IoT chips, making significant contributions to smart devices and automation.

The target board is "ESP32-P4-Function-EV". This is a development board, designed for high-performance multimedia and IoT applications. It is a state-of-the-art microcontroller board. It comes with camera, microphone, and capacitive LCD touchscreen. Also has many output ports.

The target applications for this board are:

- Visual doorbells and network cameras
- Smart home control panels and consumer appliances
- IoT dashboards and vehicle instrument clusters
- Multimedia HMI solutions



Figure 5 - ESP32-P4-Function-EV

Students will receive a total of 2 or 3 boards shipped after boards are assembled, and ready to be used.

More information on the following link:

- https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32p4/esp32-p4-function-ev-board/user_guide.html

In this proposal the application is a Coffee machine application with a focus on UI and AI. The technical implementation can be divided in three:

5.1. Frontend

The first part is the frontend, which is responsible for UI. It will be responsible for the interaction between coffee machine and the user.

The user inputs to the frontend in 3 possible ways: touches on the 7" display, the image recognition from the camera and voice command from the microphone and the frontend provides feedback to the user using the digital display.

That is the main responsibility of the frontend, to tell the user the suggestions and provide the user with the best experience possible.

The mandatory technical framework for this task is Qt for MCU. It is a graphics framework, and it allows an easy way to create the best and most performant graphics on the screen in a fast and iterative way, and it is already prepared for this board and this display. It also covers the touch driver interaction automatically.

Note the Qt for MCU documentation has multiple examples of how to use and build UI, including a coffee machine example and a camera example which are not for this specific board, but can be used as reference.

5.2. Backend

The second part is the backend, which is responsible for running the AI recognition models, interacting with the device and analyse and understand the inputs from the user.

Backend holds the business logic like which category is the user, what suggestions should it make, what are the most popular drinks, among other. It will then command the UI, providing the best suggestions based on the knowledge obtained from voice command, image recognition, etc.

The backend must use C++ and the Espressif development ESP-IDF. It is not mandatory, but we suggest using VScode with the ESP-IDF extension. The Espressif ESP-IDF also has several examples, including a camera and image recognition example.

5.3. AI Component

The third and final part is the AI component. AI is subdivided in two parts:

1. Training the model. A microcontroller is too restrictive environment to train the model for AI. They must be trained separately and integrated in the application. There are at least 2 models to be trained, the voice

command AI model and the image recognition. There are multiple tools to train the model, for example, pyTorch, TensorFlow etc. Which tool to use is up to students to decide.

- Running the model. After training the model then it needs to integrate in the application. This is called the runtime. The framework to use is decided by the students but we recommend either using with ESP-DL - <https://github.com/espressif/esp-dl> or emlearn. Both are frameworks to run AI models in Microcontrollers that supports ESP32-P4-Function-EV boards.

The following figures show the system diagram and components with development workflow:

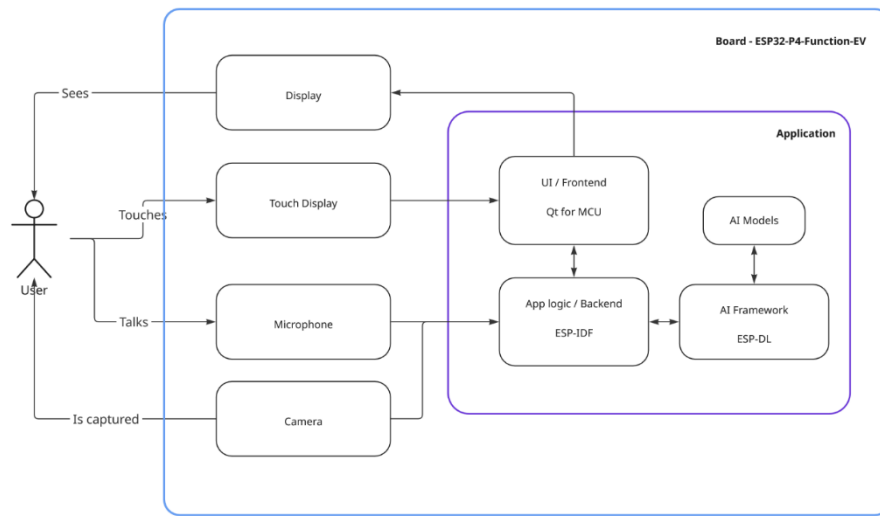


Figure 6 - System Diagram

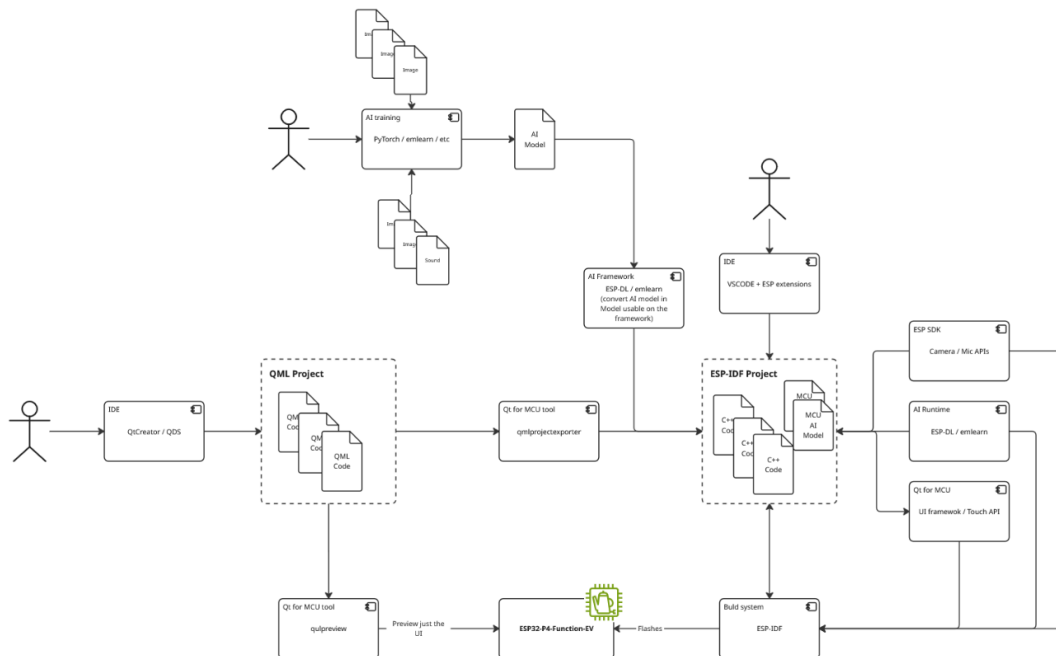


Figure 7 - Components and Development workflow